

Motivation: Why Create a Domain Model?

Motivation

I'll share a story from my experience in OO consulting and coaching. In the early 1990s, I worked with a group developing a funeral services business system in Smalltalk. I knew almost nothing about this business, so one reason to create a domain model was to understand key concepts and vocabulary quickly.

We spent about an hour sketching a UML-ish domain model, focusing on identifying key terms such as **Service** (flowers, music, etc.). These conceptual names then inspired software class names in the domain layer, keeping a low representational gap between our mental model and software.

Key Idea

- Use software class names in the domain layer inspired by names in the domain model.
- Objects contain domain-familiar information and responsibilities.
- This reduces the representational gap between stakeholders' understanding and software implementation, improving comprehension, maintainability, and time-to-development.

Illustration: Lower Representational Gap with OO Modeling

Example

Domain Model: Stakeholder's view of noteworthy concepts in the domain.

Design Model: Software classes inspired by domain model concepts (e.g., a **Payment** concept becomes a **Payment** class in code with attributes and methods).

Result: Reduced gap between mental model and software, improving comprehension and easing modifications.

Guideline: How to Create a Domain Model?

1. Identify conceptual classes relevant to the current iteration requirements.
2. Draw them as classes in a UML class diagram.
3. Add associations and attributes.

Guideline: How to Find Conceptual Classes?

Finding Conceptual Classes

A domain model shows conceptual classes. Strategies include:

1. **Reuse or Modify Existing Models:** Leverage published domain models for common domains like inventory, finance, and airline reservations.
2. **Use a Category List:** Make a list of candidate conceptual classes based on typical categories.
3. **Identify Noun Phrases:** Scan requirements, user stories, and documents for nouns representing domain concepts.

Conceptual Class Categories

Category Examples:

- **Business Transactions:** Sale, Payment, Reservation. *Guideline: Start here, since these involve money.*
- **Transaction Line Items:** SalesLineItem. *Guideline: Consider related line items next.*
- **Products/Services Related to Transactions:** Item, Flight, Seat, Meal. *Guideline: Transactions are for something.*
- **Where Transaction is Recorded:** Register, Ledger, FlightManifest. *Guideline: Important for records.*
- **Roles of People/Organizations:** Cashier, Customer, Store, Monopoly-Player, Passenger, Airline. *Guideline: Know parties involved in transactions.*
- **Place of Transaction/Service:** Store, Airport, Plane, Seat.
- **Noteworthy Events:** Sale, Payment, MonopolyGame, Flight.
- **Physical Objects:** Item, Register, Board, Piece, Die, Airplane.
- **Descriptions of Things:** ProductDescription, FlightDescription.
- **Catalogs:** ProductCatalog, FlightCatalog.
- **Containers of Things:** Store, Bin, Board, Airplane.
- **Things in a Container:** Item, Square (in a Board), Passenger.
- **Other Collaborating Systems:** CreditAuthorizationSystem, AirTraffic-Control.
- **Records:** Receipt, Ledger, MaintenanceLog.

- **Financial Instruments:** Cash, Check, LineOfCredit, TicketCredit.
- **Schedules, Manuals, Documents:** DailyPriceChangeList, RepairSchedule.

Guideline for Noun Phrase Identification: Care must be applied with this method; mechanical noun-to-class mapping isn't possible, and natural language words are often ambiguous. This technique is simple and effective for identifying candidate conceptual classes or attributes.

Practical Tips

Reusing existing models is usually fastest. Category lists help when no existing model is available. Noun phrases are always useful to identify potential classes.